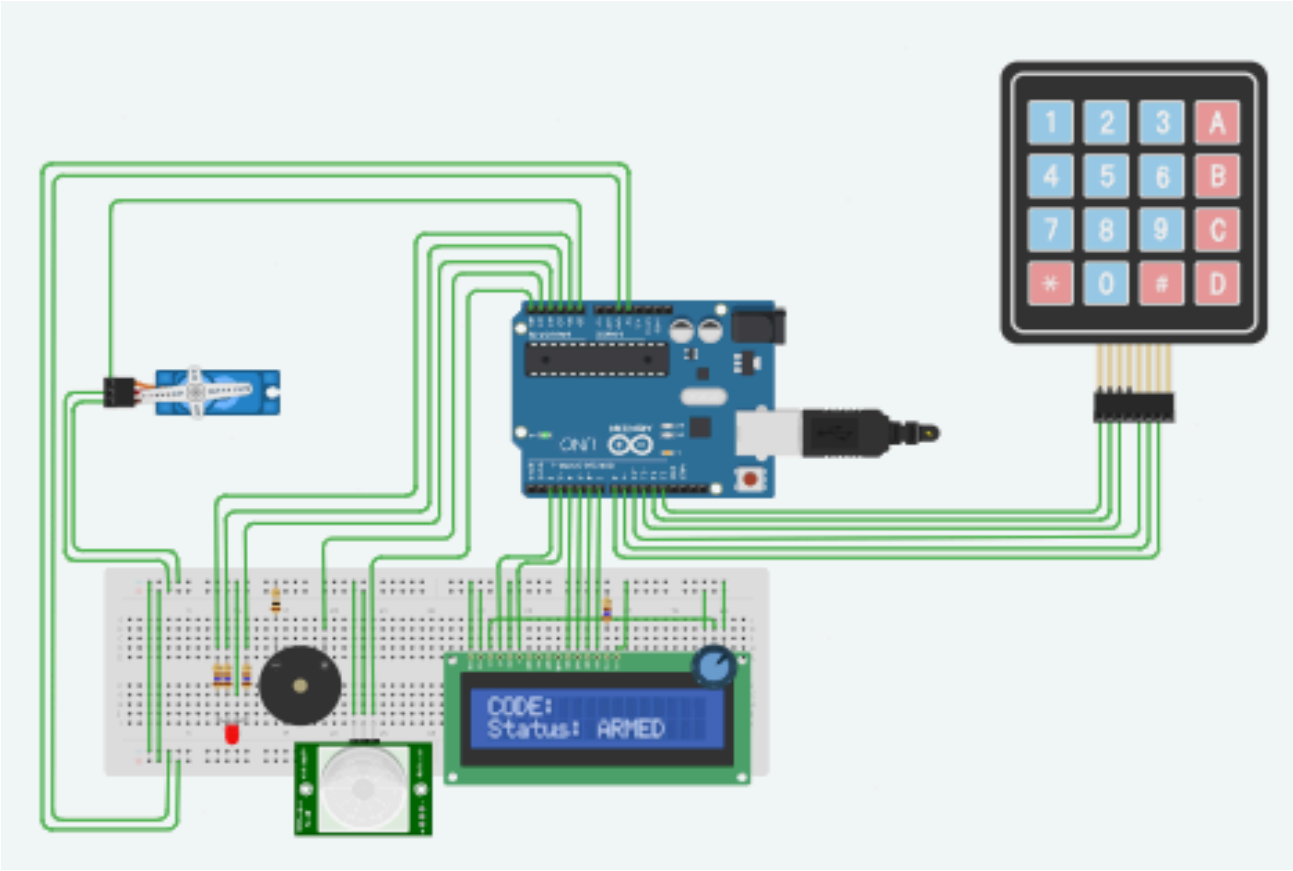
Arduino-Based Smart Home Security and Intrusion Detection System

Name	Vedant Katkhede
Roll Number	44
Class	SY-20
Enrollment Number	ADT25SOCB1492
Institute	MIT ADT University, Pune
School	School of Computing
Simulation Platform	Tinkercad
Tinkercad Link	tinkercad.com/things/6bL8bAmUEnJ-securex

Circuit diagram



1. Introduction

This project is an Arduino Uno based home security system that is armed and disarmed with a 4-digit keypad code, detects intruders with a PIR motion sensor, and responds with a locked door, a flashing light, a siren and an LCD warning.

1.1 Background

Home break-ins are a common concern, and commercial alarm systems are often expensive and closed. A microcontroller-based system gives a low-cost, programmable alternative that students can build, simulate and extend. The design is built and tested in Tinkercad Circuits, so no physical hardware is needed to verify it.

1.2 Objectives

- Design a keypad-controlled security system that can be armed and disarmed with a secret code.
- Detect unauthorised movement using a PIR motion sensor while the system is armed.
- Lock and unlock the door automatically using a servo motor.
- Give clear visual, audible and on-screen feedback using an RGB LED, a piezo buzzer and a 16x2 LCD.

- Raise the alarm after three consecutive wrong code entries.
- Verify the complete behaviour through simulation in Tinkercad.

1.3 Scope

The system covers a single entry point (one door lock and one motion sensor). The "calling security" message is a display message only; no real network call is made in this version.

2. Theory

The system works as a finite state machine with three states: UNARMED, ARMED and ALARM. A correct keypad code or a motion event moves it between states, and every state drives a fixed set of outputs.

2.1 Embedded Systems and Microcontrollers

An embedded system is a computer built into a device to perform one dedicated task. The Arduino Uno uses the ATmega328P microcontroller (16 MHz, 14 digital I/O pins, 6 analog pins, 32 KB flash, 2 KB SRAM). It repeatedly reads inputs (keypad, PIR sensor), makes decisions in software, and sets outputs (servo, LED, buzzer, LCD). This read-decide-act cycle is the core of the project.

2.2 Working Principle

1. **Power-up:** the system starts UNARMED. The door is unlocked (servo at 90 degrees), the LED is green, and the LCD shows "CODE:" and "Status: UNARMED".
2. **Arming:** the user types the 4-digit code (1235). On a correct code the state changes to ARMED, the servo moves to 0 degrees (locked), the LED turns red and the LCD shows "Status: ARMED".
3. **Intrusion detection:** while ARMED, the PIR sensor output is read continuously. A HIGH signal means motion, and the state changes to ALARM.
4. **Alarm:** the servo locks the door, the LED alternates between red and white, the piezo buzzer plays a 500 Hz tone, and the LCD shows "CALLING SECURITY / ***INTRUDER***".
5. **Disarming:** pressing any key during the alarm shows "Enter Code To Disarm". Entering the correct code stops the siren and returns the system to UNARMED.
6. **Wrong code protection:** each incorrect 4-digit entry increases a counter and shows "WRONG". After three failures the alarm triggers, even if the system was UNARMED.

2.3 State Machine

Current state	Event	Next state	Outputs
UNARMED	Correct code	ARMED	Door locked, red LED, no sound
UNARMED	3 wrong codes	ALARM	Door locked, flashing LED, siren
ARMED	Correct code	UNARMED	Door unlocked, green LED
ARMED	Motion detected (PIR HIGH)	ALARM	Door locked, flashing red/white LED, siren
ARMED	3 wrong codes	ALARM	Door locked, flashing LED, siren

Current state	Event	Next state	Outputs
ALARM	Correct code	UNARMED	Siren off, door unlocked, green LED
ALARM	Wrong code	ALARM	Siren continues

2.4 Keypad Scanning

A matrix keypad saves pins by arranging keys in rows and columns. The Keypad library drives each row in turn and reads the columns; the intersection that connects identifies the pressed key. This project uses a 3x3 portion (keys 1 to 9) of the 4x4 keypad, so only 6 Arduino pins are needed. Typed digits are collected in a String, and when its length reaches four it is compared with the stored code.

2.5 PWM and Tone Generation

Pulse Width Modulation (PWM) varies the duty cycle of a fast square wave to control average power. analogWrite() uses it to set the brightness of each colour of the RGB LED, so red, green, blue and white (all three at full) can be produced. The tone() function generates a 500 Hz square wave on the buzzer pin, and noTone() stops it. Servo position is also set by a PWM pulse, typically 1 ms to 2 ms wide, repeated every 20 ms.

2.6 Components Used

Component	Qty	Role in the project	Key facts
Arduino Uno R3	1	Central controller running the program	ATmega328P, 5 V logic, 16 MHz
PIR motion sensor	1	Detects movement of warm bodies	Digital output, HIGH on motion, 5 V supply
4x4 matrix keypad	1	Code entry (only keys 1 to 9 are used)	8 pins: 4 rows, 4 columns
16x2 LCD (HD44780)	1	Shows code input and system status	4-bit mode, 6 signal pins, contrast potentiometer
Micro servo (SG90 type)	1	Acts as the door lock	0 degrees locked, 90 degrees unlocked
Piezo buzzer	1	Siren during alarm	Driven with tone() at 500 Hz
RGB LED	1	Visual status indicator	Green = safe, red = armed, red/white flash = alarm
Resistors	several	Current limiting for LED channels and buzzer	Typically 220 ohm to 1 kilo-ohm
Breadboard and jumper wires	1 set	Common power rails and connections	Tinkercad half-size breadboard

2.7 Component Working

PIR sensor. A passive infrared sensor measures infrared radiation from its surroundings using a pyroelectric element behind a Fresnel lens. When a warm body moves across its field of view, the change in detected infrared makes the output pin go HIGH. It needs no transmitter, so it uses very little power.

Servo motor. A servo contains a DC motor, gears and a position feedback circuit. The Servo library sends the pulse width that corresponds to the requested angle. Here the horn represents the bolt of a door lock.

Piezo buzzer. A piezoelectric disc bends when voltage is applied. Feeding it an alternating square wave from tone() produces an audible sound of that frequency.

LCD 16x2. The display has 16 columns and 2 rows. In 4-bit mode it needs RS, E and four data lines (D4 to D7), and the LiquidCrystal library handles the timing. A 10 kilo-ohm potentiometer sets contrast.

2.8 Pin Connections (from the program)

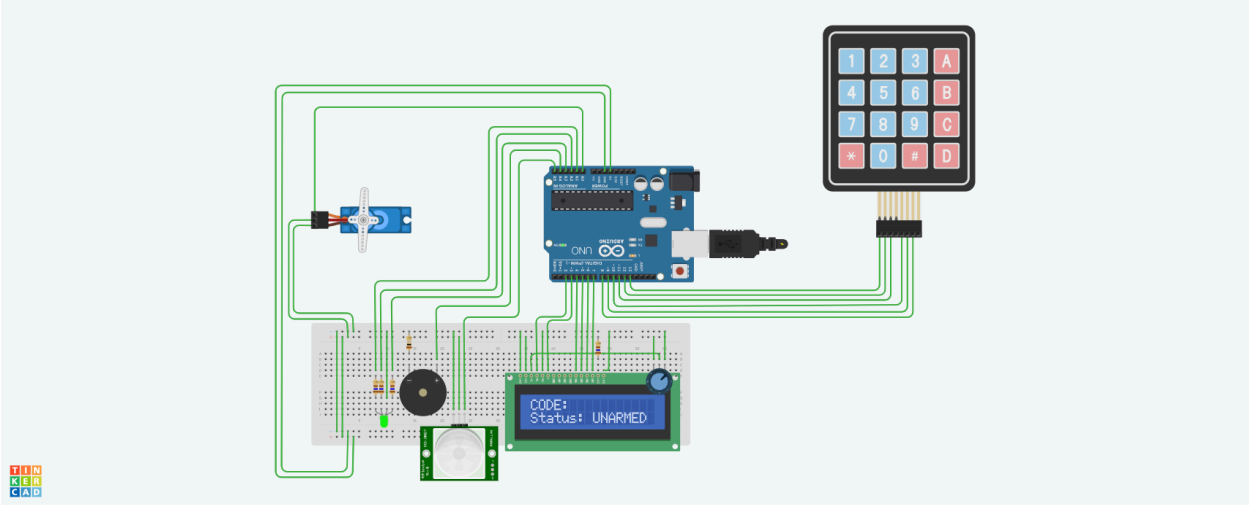
Device	Signal	Arduino pin
LCD	RS, E	2, 3
LCD	D4, D5, D6, D7	4, 5, 6, 7
Keypad	Rows 1 to 3	13, 12, 11
Keypad	Columns 1 to 3	10, 9, 8
Servo	Signal	A0
RGB LED	Green, Blue, Red	A1, A2, A3
Piezo buzzer	Signal	A4
PIR sensor	Output	A5

Analog pins A0 to A5 are used as ordinary digital pins here, which is possible on the Uno. Pins A1 to A3 are not PWM-capable, so on real hardware analogWrite() on them only gives fully on or off, which is enough for the colours used (red, green and white at full brightness).

3. Circuit Diagram and Program Source Code

3.1 Circuit Diagram (Tinkercad)

Figure 1: Tinkercad circuit of the SecureX security system.



> *Tinkercad link: https://www.tinkercad.com/things/6bL8bAmUEnJ-securex*

The circuit is built around the Arduino Uno placed at the centre. The keypad is on the right, the servo on the left, and the breadboard at the bottom carries the PIR sensor, LED, buzzer and resistors together with the LCD.

3.2 Connection Description

- **Keypad:** the row and column pins connect to digital pins 13, 12, 11 (rows) and 10, 9, 8 (columns) on the Arduino. Keys 1 to 9 are used by the program.
- **LCD:** RS and E go to pins 2 and 3, and D4 to D7 go to pins 4 to 7. VSS, RW and the LED cathode go to ground, VDD and the LED anode go to 5 V, and the contrast potentiometer is wired to V0.
- **Servo motor:** the orange signal wire goes to A0, red to 5 V and brown to ground.
- **PIR sensor:** OUT goes to A5, VCC to 5 V and GND to ground.
- **Piezo buzzer:** the positive terminal goes to A4 and the negative terminal to ground.
- **LED indicator:** the colour channels go to A1 (green), A2 (blue) and A3 (red) through current-limiting resistors, and the common terminal goes to ground.
- **Power:** the Arduino 5 V and GND pins feed the breadboard rails, which supply all the modules.

3.3 Program Source Code

```
#include <Servo.h>
#include <LiquidCrystal.h>
#include <Keypad.h>

// LCD pins
const int RS = 2;
const int E = 3;
const int DB4 = 4;
const int DB5 = 5;
const int DB6 = 6;
const int DB7 = 7;

// Servo pin
const int SERVO_PIN = A0;
```

```
// Motion sensor, alarm and RGB LED pins
const int PIR = A5;
const int ALARM = A4;
const int RED_LED = A3;
const int BLUE_LED = A2;
const int GREEN_LED = A1;

// System states
const int UNARMED = 0;
const int ARMED = 1;
const int ALARM_STATUS = 2;
int systemState = UNARMED;

int pirState;

LiquidCrystal lcd(RS, E, DB4, DB5, DB6, DB7);

// Servo positions
const int LOCKED_POS = 0;
const int UNLOCKED_POS = 90;
Servo servo;

// Keypad: only digits 1-9 (3 rows x 3 columns)
const byte ROWS = 3;
const byte COLS = 3;
const char KEYMAP[ROWS][COLS] = {
  {'1', '2', '3'},
  {'4', '5', '6'},
  {'7', '8', '9'},
};

const String CODE = "1235";
byte ROW_PINS[ROWS] = {13, 12, 11};
byte COL_PINS[COLS] = {10, 9, 8};
Keypad keypad = Keypad(makeKeymap(KEYMAP), ROW_PINS, COL_PINS, ROWS, COLS);

String input = "";
char key;
int failedAttempts = 0;
bool disarmScreen = false;

void setup() {
  lcd.begin(16, 2);
  servo.attach(SERVO_PIN);
  pinMode(PIR, INPUT);
  pinMode(ALARM, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(BLUE_LED, OUTPUT);
  updateOutputs();
}

void loop() {
  readKeypad();
```

```
  readPIR();
  updateOutputs();
}

// readKeypad: reads keypad input, arms/disarms and triggers the alarm
void readKeypad() {
  key = keypad.getKey();

  if (key != NO_KEY) {
    if (systemState == ALARM_STATUS) {
      disarmScreen = true;
    }

    input += key;

    if (input.length() == 4) {
      if (input == CODE) {
        failedAttempts = 0;
        lcd.setCursor(6, 0);
        lcd.print("CORRECT");
        delay(500);

        if (systemState == ALARM_STATUS) {
          alarmOff();
          systemState = UNARMED;
          disarmScreen = false;
        }
        else if (systemState == UNARMED) {
          systemState = ARMED;
        }
        else {
          systemState = UNARMED;
        }
      }
      else {
        failedAttempts++;
        lcd.setCursor(6, 0);
        lcd.print("WRONG");
        delay(500);

        if (failedAttempts >= 3) {
          systemState = ALARM_STATUS;
          disarmScreen = false;
          alarmOn();
        }
      }
      input = "";
    }
  }
}

// readPIR: triggers the alarm if motion is seen while armed
void readPIR() {
  pirState = digitalRead(PIR);
```

```
  if (systemState == ARMED && pirState == HIGH) {
    systemState = ALARM_STATUS;
    disarmScreen = false;
    alarmOn();
  }
}

// updateOutputs: updates LCD, servo, LED and alarm for the current state
void updateOutputs() {
  showStatus();

  if (systemState == UNARMED) {
    unlockDoor();
    displayColor(0, 255, 0);      // green
    alarmOff();
  }
  else if (systemState == ARMED) {
    lockDoor();
    displayColor(255, 0, 0);      // red
    alarmOff();
  }
  else if (systemState == ALARM_STATUS) {
    lockDoor();
    displayColor(255, 0, 0);      // red
    delay(50);
    displayColor(255, 255, 255);  // white
    delay(50);
    alarmOn();
  }
}

// displayColor: sets brightness of the RGB LED channels
void displayColor(int r, int g, int b) {
  analogWrite(RED_LED, r);
  analogWrite(GREEN_LED, g);
  analogWrite(BLUE_LED, b);
}

void lockDoor()   { servo.write(LOCKED_POS); }    // 0 degrees
void unlockDoor() { servo.write(UNLOCKED_POS); }  // 90 degrees

// showStatus: shows keypad input and status on the LCD
void showStatus() {
  if (systemState != ALARM_STATUS) {
    lcd.setCursor(0, 0);
    lcd.print("CODE: ");
    lcd.setCursor(6, 0);
    lcd.print("          ");
    lcd.setCursor(6, 0);
    lcd.print(input);
    lcd.setCursor(0, 1);
  }

  switch (systemState) {
    case UNARMED:
```

```
      lcd.print("Status: UNARMED");
      break;

    case ARMED:
      lcd.print("Status: ARMED  ");
      break;

    case ALARM_STATUS:
      if (!disarmScreen) {
        lcd.setCursor(0, 0);
        lcd.print("CALLING SECURITY");
        lcd.setCursor(0, 1);
        lcd.print(" ***INTRUDER*** ");
      }
      else {
        lcd.setCursor(0, 0);
        lcd.print("Enter Code To   ");
        lcd.setCursor(0, 1);
        lcd.print("Disarm: ");
        lcd.print(input);
        lcd.print("        ");
      }
      break;
  }

  lcd.setCursor(6, 0);
}

void alarmOn()  { tone(ALARM, 500); }   // siren on
void alarmOff() { noTone(ALARM); }      // siren off
```

3.4 Code Explanation

Part	Purpose
Libraries	Servo.h drives the lock, LiquidCrystal.h the display, Keypad.h scans the key matrix
State constants	UNARMED (0), ARMED (1) and ALARM_STATUS (2) store the current mode in systemState
setup()	Starts the LCD, attaches the servo, sets pin modes and draws the initial outputs
loop()	Repeats three tasks forever: read keypad, read PIR, update outputs
readKeypad()	Collects four digits, compares them with "1235", toggles arm/disarm, counts failures and triggers the alarm after three
readPIR()	Raises the alarm only when the system is ARMED and motion is detected

Part	Purpose
updateOutputs()	Applies the door position, LED colour and siren for the current state
displayColor()	Writes PWM values to the red, green and blue channels
showStatus()	Prints the code being typed and the status message, or the intruder and disarm screens
alarmOn() / alarmOff()	Start and stop the 500 Hz tone on the buzzer

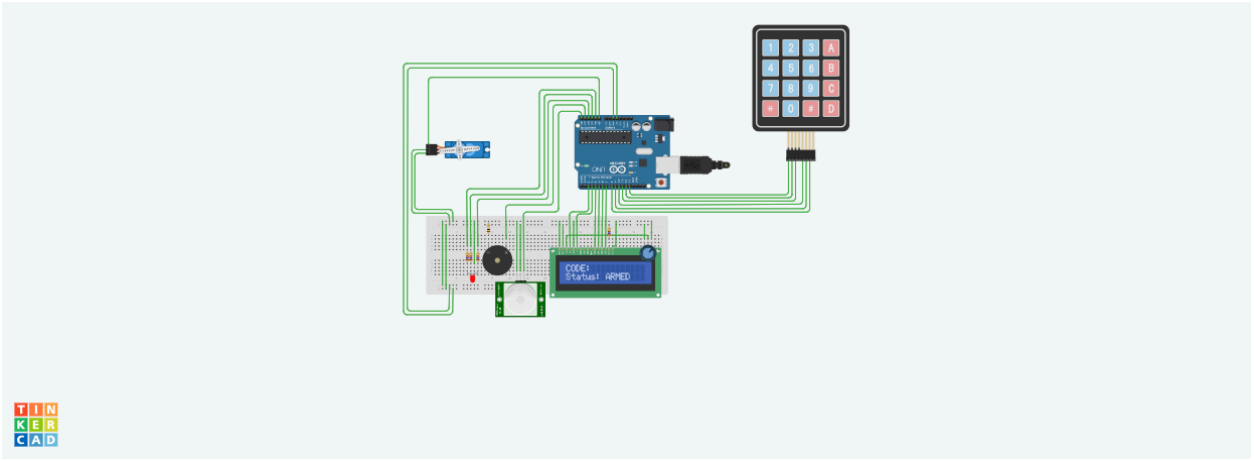
4. Results Obtained from Tinkercad

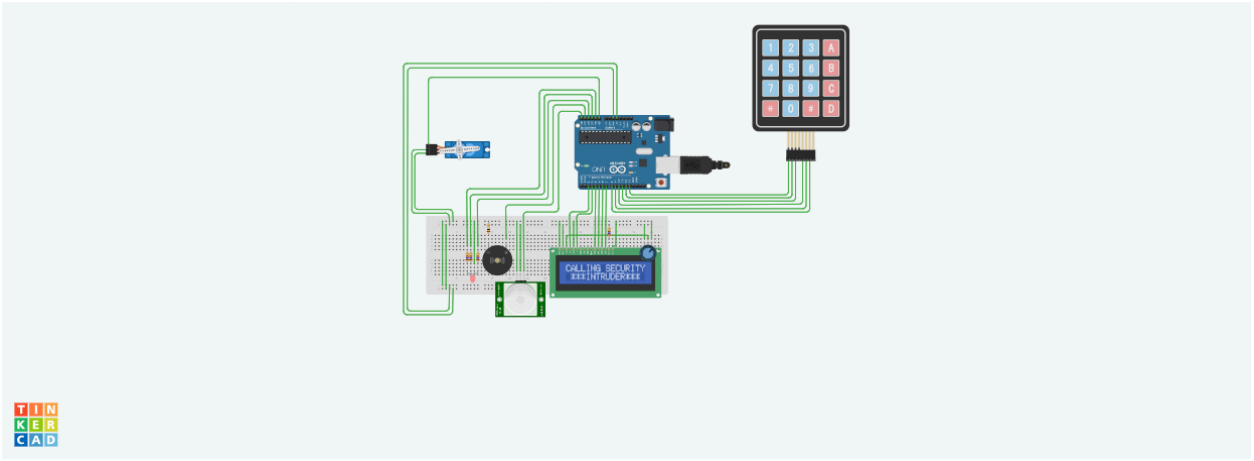
The circuit was run with Start Simulation in Tinkercad. The system boots into the UNARMED state: the LCD shows "CODE:" on the first line and "Status: UNARMED" on the second, the LED is green and the servo is at the unlocked position, as in Figure 1. The tests below cover every state change.

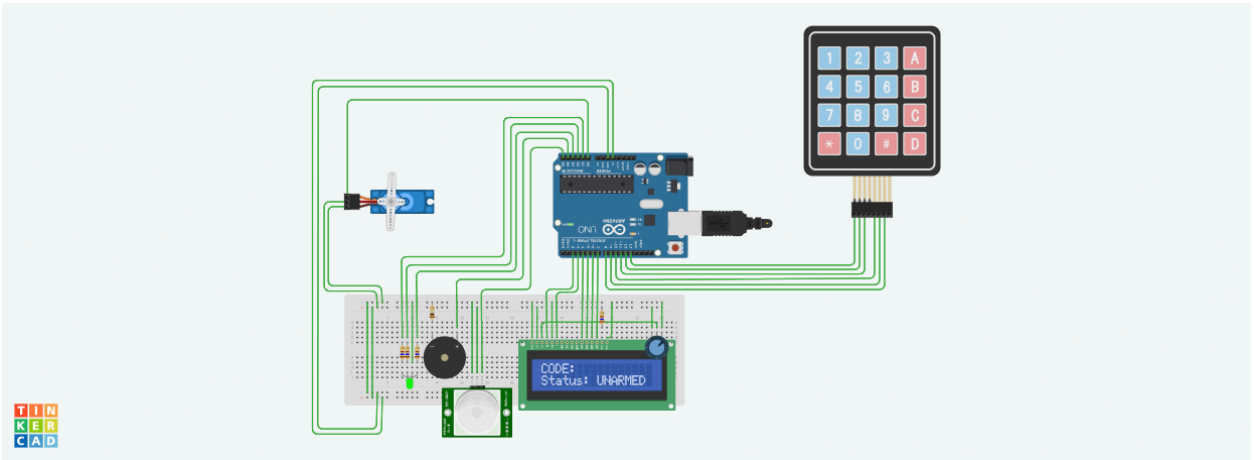
4.1 Test Cases

No.	Action in simulation	LCD display	Servo	LED	Buzzer
1	Start simulation	CODE: / Status: UNARMED	90 degrees (unlocked)	Green	Off
2	Type 1, 2, 3, 5	Digits appear after CODE:, then CORRECT, then Status: ARMED	0 degrees (locked)	Red	Off
3	Click the PIR sensor and set motion detected while ARMED	CALLING SECURITY / ***INTRUDER***	Locked	Flashing red and white	500 Hz tone on
4	Press any key during the alarm	Enter Code To Disarm: with typed digits	Locked	Flashing	On
5	Enter 1235 during the alarm	CORRECT, then Status: UNARMED	90 degrees (unlocked)	Green	Off
6	Enter a wrong code (for example 1111) once	WRONG, then back to CODE: screen	No change	No change	Off
7	Enter a wrong code three times in a row	CALLING SECURITY / ***INTRUDER***	Locked	Flashing	On
8	Enter 1235 while ARMED	CORRECT, then Status: UNARMED	Unlocked	Green	Off

4.2 Simulation Screenshots







4.3 Observations

- The state changes and outputs match the state table in Section 2.3.
- The system responds to key presses within the keypad scan time, and the PIR check runs on every pass of loop().
- The failed-attempt counter resets only after a correct code, so wrong attempts from earlier add up until then.

5. Conclusion

The Arduino-based security system was designed, programmed and verified in Tinkercad. It arms and disarms with a 4-digit code, detects motion with a PIR sensor, locks the door with a servo, and signals an intrusion through the LED, the buzzer and the LCD. All the planned states and transitions worked in simulation, so the objectives were met with low-cost parts.

5.1 Advantages

- Low cost and easy to build from common components.
- Clear feedback through the LCD, LED and siren.
- Code-based access with lockout alarm after three wrong attempts.
- Simple state-based code that is easy to modify.

5.2 Limitations

- The code is fixed in the program and cannot be changed without re-uploading.
- Only one PIR sensor and one door are covered.
- The keypad code is not stored in non-volatile memory and the system has no battery backup.

5.3 Future Scope

- Add a GSM or Wi-Fi module (ESP8266) to send SMS or app notifications.
- Store and change the passcode in EEPROM.
- Add door/window magnetic sensors, a camera module and fire/gas sensors.
- Replace delay() with millis()-based timing and add a battery backup.

References

1. Arduino documentation: Servo, LiquidCrystal and tone() references, arduino.cc.
2. Keypad library for Arduino, Mark Stanley and Alexander Brevig.
3. Autodesk Tinkercad Circuits, tinkercad.com.
4. ATmega328P datasheet, Microchip Technology.
5. Project simulation: https://www.tinkercad.com/things/6bL8bAmUEnJ-securex